Design Document: RBAC v2 Service Account Authentication Migration

Host-Based Inventory

Author: Muhammad Arif **Created:** April 7, 2026 **Last Updated:** April 30, 2026 **Status:** IMPLEMENTED **JIRA:** RHINENG-25611 **Related:** RHCLOUD-43063 (PSK Security Vulnerability), RHCLOUD-45759

Abstract

This document describes the migration of Host-Based Inventory's RBAC v2 workspace API integration from Pre-Shared Key (PSK) and x-rh-identity-only authentication to OAuth2 service account authentication. This migration addresses a critical security vulnerability (RHCLOUD-43063) and standardizes authentication patterns across HBI's RBAC integrations.

PSK was originally added on April 30, 2025 (commit `9c002439`, RHINENG-17735). This migration replaces it exactly one year later.

Overview

Goal: Replace PSK and x-rh-identity-only authentication with OAuth2 service account authentication for all RBAC v2 workspace API calls.

Key Driver: Remove PSK security vulnerability (RHCLOUD-43063)

Scope: All RBAC v2 workspace functions in `lib/middleware.py` and their callers.

Changes from Master

Files Modified

File	Changes
`lib/middleware.py`	Added token utilities, updated header builder, removed PSK, added `_build_service_account_headers()`, added `get_rbac_workspace_as_service()`
`lib/group_repository.py`	Updated import and call to use `get_rbac_workspace_as_service()`
`tests/fixtures/api_fixtures.py`	Updated `enable_rbac` fixture to mock OAuth2 token

File	Changes
`tests/test_rbac.py`	Added 10 new tests for OAuth2 auth, service account headers, PSK removal
`tests/test_host_mq_service.py`	Updated MQ tests from PSK to OAuth2 assertions
`tests/test_models.py`	Updated mock target for renamed function
`tests/test_api_groups_get.py`	Added OAuth2 token mock for 3 tests

---

1. Token Management Utilities (NEW)

File: `lib/middleware.py`

Reuses `OAuth2ClientCredentials` from the `kessel-sdk` library directly, consistent with how Kessel itself authenticates. No custom wrapper class needed.

`_get_rbac_oauth_client()` – Singleton OAuth2 Client

```python
from kessel.auth import OAuth2ClientCredentials, fetch_oidc_discovery

_rbac_oauth_client = None  # Global singleton

def _get_rbac_oauth_client() -> OAuth2ClientCredentials:
    global _rbac_oauth_client
    if _rbac_oauth_client is None:
        config = inventory_config()
        discovery = fetch_oidc_discovery(config.kessel_auth_oidc_issuer)
        _rbac_oauth_client = OAuth2ClientCredentials(
            client_id=config.kessel_auth_client_id,
            client_secret=config.kessel_auth_client_secret,
            token_endpoint=discovery.token_endpoint,
        )
        logger.info("RBAC OAuth2 client initialized", extra={...})
    return _rbac_oauth_client
```

`_get_rbac_access_token()` – Token Fetch with Caching

```python
def _get_rbac_access_token() -> str:
    oauth_client = _get_rbac_oauth_client()
    try:
        token_response = oauth_client.get_token()
        return token_response.access_token  # Returns RefreshTokenResponse object
    except Exception as e:
```

```
        logger.exception("Failed to get RBAC access token", extra={"error": str(e)})
        raise
```

Key implementation details: - Reuses same `OAuth2ClientCredentials` class as Kessel - No custom token caching – `kessel-sdk 2.6.0` handles caching internally with a 300-second refresh buffer - Uses same service account credentials (`KESSEL_AUTH_CLIENT_ID`, `KESSEL_AUTH_CLIENT_SECRET`) - `get_token()` returns a `RefreshTokenResponse` object (not a dict) – access via `.access_token` attribute - OIDC discovery is performed once at singleton creation time - Error logging included for token fetch failures

---

2. Request Header Builders

File: `lib/middleware.py`

`_build_rbac_request_headers()` – Modified Added `use_service_account` parameter (default: `True`). All existing callers automatically get service account auth without code changes.

```
def _build_rbac_request_headers(
    identity_header: str | None = None,
    request_id_header: str | None = None,
    use_service_account: bool = True,  # NEW
) -> dict:
    request_headers = {
        IDENTITY_HEADER: identity_header or request.headers[IDENTITY_HEADER],
        REQUEST_ID_HEADER: request_id_header or request.headers.get(REQUEST_ID_HEADER),
    }
    if use_service_account:
        access_token = _get_rbac_access_token()
        request_headers["Authorization"] = f"Bearer {access_token}"
    return request_headers
```

Impact: All functions calling `_build_rbac_request_headers()` automatically get service account auth. This includes `post_rbac_workspace()`, `get_rbac_workspaces()`, `get_rbac_workspaces_by_ids()`, `delete_rbac_workspace()`, and `patch_rbac_workspace()`.

`_build_service_account_headers()` – NEW Builds headers for service-to-service calls that run **outside Flask request context** (e.g., MQ service). Does not access `request.headers`.

```
def _build_service_account_headers(org_id: str) -> dict:
    access_token = _get_rbac_access_token()
    return {
        "Authorization": f"Bearer {access_token}",
```

```
        "X-RH-RBAC-ORG-ID": org_id,
        "X-RH-RBAC-CLIENT-ID": "inventory",
    }
```

Used by: - `rbac_create_ungrouped_hosts_workspace()` – called from MQ service during host ingestion - `get_rbac_workspace_as_service()` – called for write-without-read permission scenarios

---

3. `rbac_create_ungrouped_hosts_workspace()` – PSK Removed

File: `lib/middleware.py`

Before (PSK):

```
psk = inventory_config().rbac_psk
request_headers = {
    "X-RH-RBAC-PSK": psk,              # REMOVED - Security vulnerability
    "X-RH-RBAC-ORG-ID": identity.org_id,
    "X-RH-RBAC-CLIENT-ID": "inventory",
}
```

After (OAuth2 service account):

```
request_headers = _build_service_account_headers(identity.org_id)
```

Important: This function is called from both Flask request context (API) and outside request context (MQ service), so it must NOT access `request.headers`. The `_build_service_account_headers()` helper satisfies this constraint.

---

4. `get_rbac_workspace_as_service()` – NEW

File: `lib/middleware.py`

New function that fetches a workspace using service account auth with explicit `org_id`. Solves the write-without-read permission defect (RHINENG-25822): when a user has `inventory:groups:write` but not `inventory:groups:read`, the read-back after write operations would fail because `get_rbac_workspace_by_id()` uses the user's identity.

```
def get_rbac_workspace_as_service(
    workspace_id: str, org_id: str
) -> dict[str, Any]:
    request_headers = _build_service_account_headers(org_id)
    if workspaces := get_rbac_workspaces_by_ids([workspace_id], request_headers=request_head
        return workspaces[0]
    else:
        raise ResourceNotFoundException(f"Workspace {workspace_id} not found")
```

Used by: `lib/group_repository.py` for ungrouped workspace read-back after creation.

---

5. Functions Requiring No Code Changes

These functions already call `_build_rbac_request_headers()` and automatically get service account auth via the new `use_service_account=True` default:

Function	File	Endpoint
`post_rbac_workspace()`	`lib/middleware.py`	`POST /api/rbac/v2/workspaces/`
`get_rbac_workspaces()`	`lib/middleware.py`	`GET /api/rbac/v2/workspaces/`
`get_rbac_workspaces_by_ids()`	`lib/middleware.py`	`GET /api/rbac/v2/workspaces/?ids=...`
`delete_rbac_workspace()`	`lib/middleware.py`	`DELETE /api/rbac/v2/workspaces/{id}`
`patch_rbac_workspace()`	`lib/middleware.py`	`PATCH /api/rbac/v2/workspaces/{id}`

Authentication Pattern

Before (PSK) – Security Vulnerability

```
Headers:
  X-RH-RBAC-PSK: <static-key>       # Removed
  X-RH-RBAC-ORG-ID: <org_id>
```

Before (User Operations) – Missing Service Account

```
Headers:
  x-rh-identity: <user_jwt>          # User context only, no service auth
```

After (User Operations via Flask Request Context)

```
Headers:
  Authorization: Bearer <token>      # Service account (proves HBI is authorized)
  x-rh-identity: <user_jwt>          # User context (permission scoping)
  x-rh-insights-request-id: <id>     # Request tracing
```

After (Service Operations Outside Flask Context)

```
Headers:
  Authorization: Bearer <token>      # Service account
  X-RH-RBAC-ORG-ID: <org_id>        # Org context
  X-RH-RBAC-CLIENT-ID: inventory     # Client identifier
```

Performance Analysis

No performance degradation in production.

Aspect	Impact
OIDC Discovery	One-time cost at process startup (~100-300ms)
Token fetch	Cached by `kessel-sdk 2.6.0`; refresh only when token nears expiry (300s buffer)
Per-request overhead	Cached token lookup (microseconds) replaces config lookup (microseconds)
Cold start	~200-500ms penalty on first request after process startup
Token refresh	~50-200ms every ~55 minutes (one request pays this cost)

Reused Infrastructure

No new credentials or services required.

Component	Source
`KESSEL_AUTH_CLIENT_ID`	Already configured for Kessel
`KESSEL_AUTH_CLIENT_SECRET`	Already configured for Kessel
`KESSEL_AUTH_OIDC_ISSUER`	Already configured for Kessel
`kessel.auth.OAuth2ClientCredentials`	From `kessel-sdk 2.6.0`
`kessel.auth.fetch_oidc_discovery`	From `kessel-sdk 2.6.0`

Test Results

- **2593 tests passed**, 0 failed, 4 skipped, 2 xfailed
- **`make style`**: All linters pass (ruff, mypy, pre-commit)

References

- **JIRA:** RHINENG-25611
- **Security:** RHCLOUD-43063 (PSK vulnerability)
- **Related:** RHCLOUD-45759
- **PSK original commit:** `9c002439` (April 30, 2025, RHINENG-17735)
- **Branch:** `use-service-account`

Last Updated: April 30, 2026